BATCH 26

Inceptez Data Science Presents

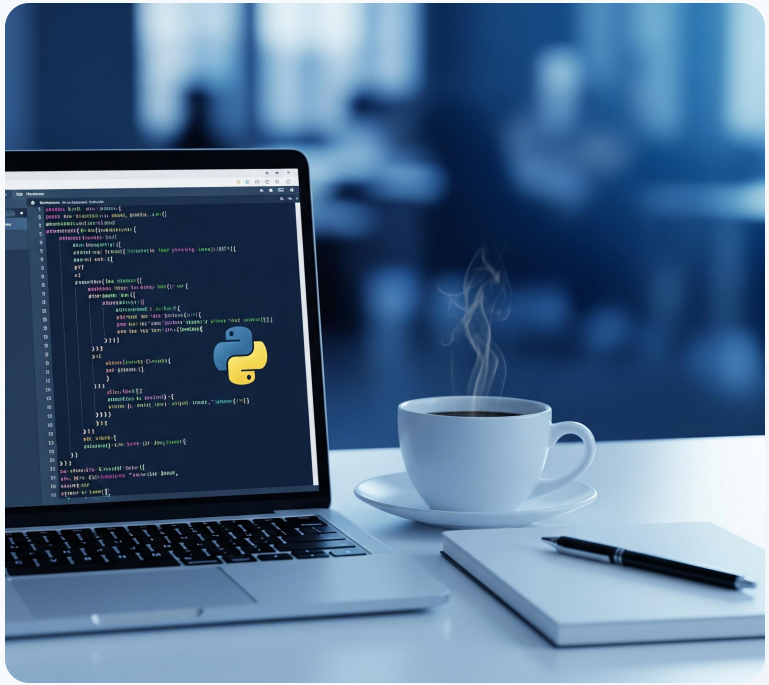
Python Day - 3

Laxmi Narayen Nagarajan Venkatesan

25th-April-2026

REVIEW

Reflecting on Our Last Sessions



Review

Data Types — The Five Basic Flavors

1. Integer (int)

"Count Anything"

Whole numbers, exact quantities.

Ex: 11,000 students, 10 months.

2. Float (float)

"Measure Anything"

Precision numbers with decimals.

Ex: 4.8 rating, 85.0% hands-on.

3. Boolean (bool)

"Yes or No"

Simplest type for logic decisions.

Values: True / False

4. String (str)

"Letters & Text"

Wrapped in quotes to identify as text.

Ex: "inceptez", "2026"

5. Complex

"AI Research"

Advanced math with imaginary parts.

Ex: 3 + 5j

💡 Analogy: Ingredients vs. Recipe

Think of data types as categories of ingredients. You wouldn't use a "string" (text) to perform "math" (multiplication) any more than you'd use a spoon to cut an onion. Handling data correctly requires knowing its **flavor**.

👩‍🏫 Lab Activity

What's My Type?

```
1. "11/04/2026"
2. 12
3. 85.0
4. True
```

Instructor Ans: 1.Str, 2.Int, 3.Float, 4.Bool

REVIEW

Data Type Casting — The Art of Conversion

Implicit Casting (Automatic)

Python automatically converts one data type to another without user involvement to avoid data loss.

```
x = 10 + 5.5 # Result: 15.5 (float)
```

Explicit Casting (Manual)

User manually converts data types using built-in constructor functions:

- **int()**: Constructs an integer from a float or string.
- **float()**: Constructs a float from an integer or string.
- **str()**: Constructs a string from any type.

Example:

```
age = int("25") # String to Int
price = float(10) # Int to Float
```

💡 Tip: Python cannot cast non-numeric strings to integers (e.g., int("Hello") results in a ValueError).

REVIEW

The Data Structure "DNA" Comparison

🟦 List

Type: `[ ]`

Heterogeneity: **Supported**

Index: **Supported (0)**

Mutability: **Mutable via index**
Mutability means ability to change the values of the data structure

🔒 Tuple

Type: `( )`

Heterogeneity: **Supported**

Index: **Supported**

Mutability: **Immutable**

📖 Dictionary

Type: `{ k:v }`

Heterogeneity: **Supported**

Index: Key-based

Mutability: **Mutable**

🧪 Set

Type: `{ }`

Heterogeneity: **Supported**

Index: No Position

Mutability: **No change via index**

The "Four-Point Check" Strategy

1. Type (Brackets), 2. Heterogeneity (Mixed data?), 3. Index (Position number?), 4. Mutability (Can it change?).

REVIEW

The Data Structure "DNA" Comparison

🟦 List

Type: [ ]

Heterogeneity: Supported

Index: Supported (0)

Mutability: Mutable

🔒 Tuple

Type: ( )

Heterogeneity: Supported

Index: Supported

Mutability: Immutable

📖 Dictionary

Type: { k:v }

Heterogeneity: Supported

Index: Key-based

Mutability: Mutable

🧪 Set

Type: { }

Heterogeneity: Supported

Index: No Position

Mutability: Mutable

The "Four-Point Check" Strategy

1. Type (Brackets), 2. Heterogeneity (Mixed data?), 3. Index (Position number?), 4. Mutability (Can it change?).

REVIEW

The Data Structure "DNA" Comparison

🟦 List

Type: `[ ]`

Heterogeneity: **Supported**

Index: **Supported (0)**

Mutability: **Mutable**

🔒 Tuple

Type: `( )`

Heterogeneity: **Supported**

Index: **Supported**

Mutability: **Immutable**

📖 Dictionary

Type: `{ k:v }`

Heterogeneity: **Supported**

Index: Key-based

Mutability: **Mutable**

🧪 Set

Type: `{ }`

Heterogeneity: **Supported**

Index: No Position

Mutability: **Mutable**

The "Four-Point Check" Strategy

1. Type (Brackets), 2. Heterogeneity (Mixed data?), 3. Index (Position number?), 4. Mutability (Can it change?).

REVIEW

Python Tuple Operations

Indexing & Slicing

Identical to lists, access data via position or range.

Access: `tup[0]` or `tup[-1]`
Slice: `tup[1:3]`

Combination & Repetition

Join tuples or repeat elements to create new tuples.

Concatenate: `t1 + t2`
Multiply: `t1 * 3`

Built-in Tuple Methods

Tuples only have two methods due to their immutability.

- **count(x):** Returns number of times x appears.
- **index(x):** Finds the first index of value x.

💡 **Pro Tip:** Since tuples are immutable, "operations" like addition actually return a brand new tuple object.

PART 1

The Data Structure "DNA" Comparison

🟦 List

Type: [ ]

Heterogeneity: Supported

Index: Supported (0)

Mutability: Mutable

🔒 Tuple

Type: ( )

Heterogeneity: Supported

Index: Supported

Mutability: Immutable

📖 Dictionary

Type: { k:v }

Heterogeneity: Supported

Index: Key-based

Mutability: Mutable

🧪 Set

Type: { }

Heterogeneity: Supported

Index: No Position

Mutability: Mutable

The "Four-Point Check" Strategy

1. Type (Brackets), 2. Heterogeneity (Mixed data?), 3. Index (Position number?), 4. Mutability (Can it change?).

ANALOGY

The "Everyone Greets Everyone" Problem

Why it gets hard so fast ($O(n^2)$)

Imagine a room full of friends. To be super polite, every single person wants to look at and wave to every other person.

- **2 Friends:** Only 1 wave happens.
- **5 Friends:** You need 10 waves!
- **10 Friends:** Suddenly, that's 45 waves!

As the group gets bigger, the "looking and waving" work explodes like a rocket!

```
"Each one looks at every one"
```

💡 **Thinking Rule:** In smart machines, "paying notice" to every word at once is why they need so much power—it's like a huge party where everyone is talking to everyone else!

CONCEPT

Visualizing Time Complexity

O(1) - Constant

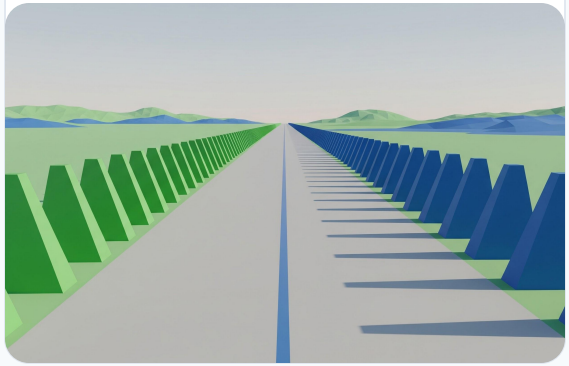
No matter the size, it takes the same time. Instant access.

`"Just picking one"`

O(n) - Linear

Time grows steadily with data size. Like reading a book.

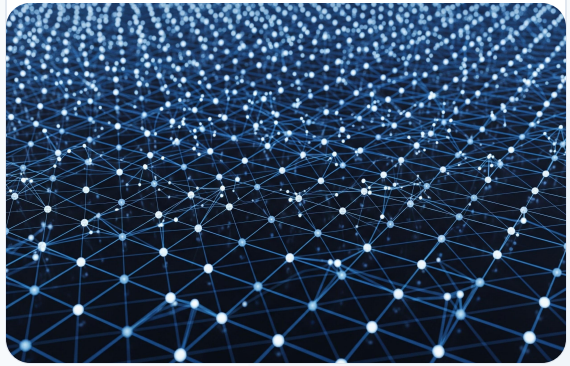
`"Checking every item"`



O(n^2) - Quadratic

Time explodes as data grows. Every item meets every item.

`"Nested interactions"`



💡 **Pro Tip:** Think of it as "how much work" you do as your data pile gets bigger. Constant (1) stays flat; Linear (n) is a slope; Quadratic (n^2) is a rocket.

PART 1

Python Set Operations

Mathematical Operations

Sets excel at unique membership and comparisons.

Union: `s1 | s2` or `s1.union(s2)`

Intersection: `s1 & s2`

Difference & Symmetric Diff

Identify elements unique to specific sets.

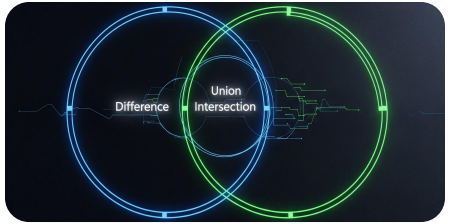
Difference: `s1 - s2`

Symmetric: `s1 ^ s2`

Common Set Methods

Unlike tuples, sets are mutable and support modifications.

- **add(x):** Adds an element to the set.
- **remove(x):** Removes x; raises error if not found.



💡 **Pro Tip:** Sets are unordered; you cannot access elements via index like lists or tuples.

CONCEPT

Set Theory: The Foundation of AI Logic

From Mathematics to AI

Set theory provides the formal language for defining membership and logical relations in Machine Learning.

- **Probability:** Defining sample spaces and events as sets.
- **Data Filtering:** Identifying unique instances and duplicates.
- **Knowledge Graphs:** Relations between entities (triples).

Python Set Operations

Python's `set` type implements set theory natively for high-performance membership testing.

Union: `s1 | s2` (Combine unique elements)

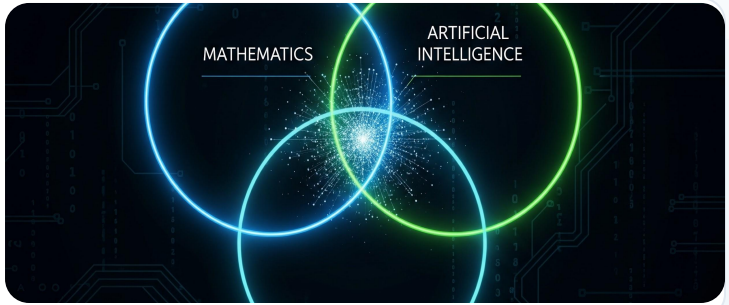
Intersection: `s1 & s2` (Common elements)

Difference: `s1 - s2` (Elements in s1 but not s2)

Practical AI Application: Venn Logic

In information retrieval (RAG), set operations are used to refine document retrieval by intersecting relevant keyword sets.

Sets are unordered and unique, making them the ideal structure for tracking feature vocabularies and categorical labels in large-scale ML pipelines.



💡 **Pro Tip:** Use sets instead of lists for membership checks (`x in s`) as it is O(1) complexity, significantly speeding up data preprocessing.

PART 2

The Data Structure "DNA" Comparison

🟦 List

Type: [ ]

Heterogeneity: Supported

Index: Supported (0)

Mutability: Mutable

🔒 Tuple

Type: ( )

Heterogeneity: Supported

Index: Supported

Mutability: Immutable

📖 Dictionary

Type: { k:v }

Heterogeneity: Supported

Index: Key-based

Mutability: Mutable

🧪 Set

Type: { }

Heterogeneity: Supported

Index: No Position

Mutability: Mutable

The "Four-Point Check" Strategy

1. Type (Brackets), 2. Heterogeneity (Mixed data?), 3. Index (Position number?), 4. Mutability (Can it change?).

PART 2

Python Dictionary Operations

Key-Value Management

Access, update, or add pairs using unique keys.

Access: `dict["key"]` or `.get()`
Update/Add: `dict["k"] = "v"`

Dictionary View Objects

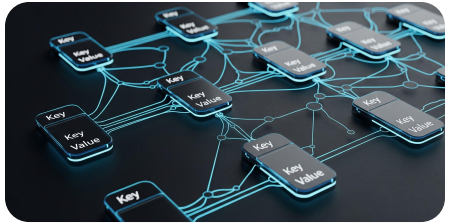
Dynamically view keys, values, or items.

Keys: `.keys()`
Values: `.values()`
Pairs: `.items()`

Core Methods & Cleanup

Dictionaries provide robust tools for safe data removal.

- **pop(key):** Removes key and returns its value.
- **update(dict2):** Merges two dictionaries together.



💡 **Pro Tip:** Since Python 3.7+, dictionaries maintain insertion order for all keys.

PART 3

Data Structure Type Casting

Standard Casting Functions

Convert between sequences using constructor functions.

To List: `list(iterable)`

To Tuple: `tuple(iterable)`

To Set: `set(iterable)`

Special Case: Dictionaries

Requires paired data (key-value) to cast successfully.

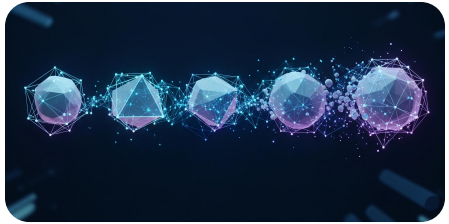
From Pairs: `dict([('a', 1), ('b', 2)])`

Zip: `dict(zip(keys, values))`

Common Conversion Scenarios

Data transformation is essential for optimizing performance or ensuring data safety.

- **Deduplication:** Cast a list to a `set()` to remove duplicates.
- **Freezing:** Cast a list to a `tuple()` to make it immutable.
- **Sorting:** Cast sets/dicts to a `list()` to apply custom sorts.



💡 **Pro Tip:** Casting from a Dictionary directly to a List/Set only retrieves the `keys` by default.

PART 4

Control Flow: Loops & Conditionals

Conditional Control

Logic gates that execute code based on specific truths.

Structure: `if`, `elif`, `else`

Usage: Branches the program path based on Boolean results.

Iterative Loops

Repeating a block of code multiple times until a condition is met.

Keyword: `for` (sequence) or `while` (condition)

Goal: Automate repetitive tasks efficiently across data sets.

Looping Strategies

Choosing the right tool for the job ensures clean, readable code.

- **For Loops:** Ideal for iterating over lists, strings, or ranges.
- **While Loops:** Used when the number of iterations is unknown.

💡 **Pro Tip:** Always ensure a "break" condition in `while` loops to avoid infinite execution.

PART 5

Optimizing Looping Strategies

For Loops: Definite Iteration

Ideal for iterating over known sequences like `lists`, `strings`, or `ranges`. Use when the number of steps is pre-determined.

While Loops: Indefinite Iteration

Used when the number of iterations is unknown. Executes as long as a condition remains `True`. Perfect for event listening or state-based logic.

💡 **Pro Tip:** Choosing the right tool for the job ensures clean, readable, and performant code.